

✓ Project: Smart Fraud Detection Pipeline

Silver Layer – Data Cleaning and Enrichment

Author: Snehal A. Bhosale

College: Sanjivani College of Engineering, Kopargaon

Email: snehalbhosale1807@gmail.com

Student ID: CT_CSI_DE_1177

Technology: PySpark, Spark SQL, Parquet/Delta Lake

Objective:

To clean and validate raw data from the Bronze layer by handling null values, duplicates, invalid dates, invalid amounts, whitespace, negative values, and orphan records. The cleaned datasets are then enriched by joining transactions with account information.

✓ Step 1: Import Libraries

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col,
    trim,
    to_date,
    coalesce,
    when,
    lit,
    current_timestamp,
    try_to_timestamp # Added try_to_timestamp
)
```

✓ Step 2: Upload Output Files of Bronze Layer

```
from google.colab import files

uploaded = files.upload()
```

No files selected.

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving bronze accounts.csv to bronze accounts.csv

```
-----  
Saving bronze_fraud_watchlist.csv to bronze_fraud_watchlist.csv  
Saving bronze_transactions.csv to bronze_transactions.csv
```

```
import os
```

```
print(os.listdir("/content"))
```

```
['.config', 'bronze_accounts.csv', 'bronze_transactions.csv', 'bronze_fraud_w
```

✓ Step 2: Start Spark

```
spark = (  
    SparkSession.builder  
        .appName("SmartFraudDetection-Silver")  
        .getOrCreate()  
)
```

```
print("Spark Version:", spark.version)
```

```
Spark Version: 4.0.3
```

✓ Step 3: Read Bronze Data

```
accounts = (  
    spark.read  
        .option("header", True)  
        .option("inferSchema", False)  
        .csv("bronze_accounts.csv")  
)  
  
transactions = (  
    spark.read  
        .option("header", True)  
        .option("inferSchema", False)  
        .csv("bronze_transactions.csv")  
)  
  
fraud_watchlist = (  
    spark.read  
        .option("header", True)  
        .option("inferSchema", False)  
        .csv("bronze_fraud_watchlist.csv")  
)
```

✓ Step 4: Check Raw Counts

```
print("Accounts:", accounts.count())
print("Transactions:", transactions.count())
print("Fraud Watchlist:", fraud_watchlist.count())
```

```
Accounts: 505
Transactions: 20011
Fraud Watchlist: 46
```

✓ Step 5: Inspect Schemas

```
accounts.show(5, truncate=False)
transactions.show(5, truncate=False)
fraud_watchlist.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
|account_id|customer_name|account_type|credit_limit|branch|ingestion_timestam
+-----+-----+-----+-----+-----+-----+
|ACC0059   |Customer_59  |Salary     |500000     |Pune  |2026-08-08T12:43:1
|ACC0139   |Customer_139 |Current    |100000     |Delhi |2026-08-08T12:43:1
|ACC0182   |Customer_182 |Salary     |200000     |Pune  |2026-08-08T12:43:1
|ACC0302   |Customer_302 |Savings    |500000     |Mumbai|2026-08-08T12:43:1
|ACC0254   |Customer_254 |Savings    |200000     |Delhi |2026-08-08T12:43:1
+-----+-----+-----+-----+-----+-----+
```

only showing top 5 rows

```
+-----+-----+-----+-----+-----+-----+
|txn_id   |account_id|txn_date  |amount    |merchant      |ingestion_timestamp
+-----+-----+-----+-----+-----+-----+
|TXN009314|ACC0332   |2025-01-10|107549.32|Swiggy        |2026-08-08T12:43:11
|TXN013000|ACC0412   |2025-03-17|22297.68 |Zomato        |2026-08-08T12:43:11
|TXN018018|ACC0486   |2025-05-10|115208.63|Zomato        |2026-08-08T12:43:11
|TXN013757|ACC0443   |2025-07-24|45099.7  |ATM-Withdrawal|2026-08-08T12:43:11
|TXN016494|ACC0062   |2025-02-25|70884.04 |Swiggy        |2026-08-08T12:43:11
+-----+-----+-----+-----+-----+-----+
```

only showing top 5 rows

```
+-----+-----+-----+-----+-----+-----+
|account_id|fraud_type      |flagged_date|ingestion_timestamp      |source_sys
+-----+-----+-----+-----+-----+-----+
|ACC0116   |Account Takeover|2025-05-26  |2026-08-08T12:43:12.516Z|fraud_watc
|ACC0300   |Card Skimming   |10-Feb-2025 |2026-08-08T12:43:12.516Z|fraud_watc
|ACC0280   |Account Takeover|2025-03-03  |2026-08-08T12:43:12.516Z|fraud_watc
|ACC0464   |Account Takeover|2025-07-14  |2026-08-08T12:43:12.516Z|fraud_watc
|ACC0072   |Account Takeover|2025-05-17  |2026-08-08T12:43:12.516Z|fraud_watc
+-----+-----+-----+-----+-----+-----+
```

only showing top 5 rows

✓ Step 5: Clean Accounts

```
accounts_clean = (
    accounts
```

```
.withColumn("account_id", trim(col("account_id")))
.withColumn("customer_name", trim(col("customer_name")))
.withColumn("account_type", trim(col("account_type")))
.withColumn("branch", trim(col("branch")))
.withColumn("credit_limit", col("credit_limit").cast("double"))
)
```

✓ Step 6: Remove Invalid Accounts

```
accounts_silver = (
    accounts_clean
    .filter(col("account_id").isNotNull())
    .filter(col("account_id") != "")
    .filter(col("customer_name").isNotNull())
    .filter(col("customer_name") != "")
    .filter(col("branch").isNotNull())
    .filter(col("branch") != "")
    .filter(col("credit_limit").isNotNull())
    .filter(col("credit_limit") >= 0)
    .dropDuplicates(["account_id"])
)
```

```
print("Accounts before cleaning:", accounts.count())
print("Accounts after cleaning :", accounts_silver.count())
```

```
Accounts before cleaning: 505
Accounts after cleaning : 501
```

✓ Step 7: Clean Transactions

```
transactions_clean = (
    transactions
    .withColumn("txn_id", trim(col("txn_id")))
    .withColumn("account_id", trim(col("account_id")))
    .withColumn("txn_date", trim(col("txn_date")))
    .withColumn("amount", trim(col("amount")))
    .withColumn("merchant", trim(col("merchant")))
)
```

✓ Step 9: Convert Amount

```
from pyspark.sql.functions import expr

transactions_clean = transactions_clean.withColumn(
```

```

        "amount",
        expr("try_cast(amount AS DOUBLE)")
    )

```

✓ Step 10: Handle Multiple Date Formats

```

from pyspark.sql.functions import (
    col, trim, coalesce, to_date, lit, expr
)

transactions_clean = (
    transactions
    .withColumn("txn_id", trim(col("txn_id")))
    .withColumn("account_id", trim(col("account_id")))
    .withColumn("txn_date_raw", trim(col("txn_date")))
    .withColumn("amount_raw", trim(col("amount")))
    .withColumn("merchant", trim(col("merchant")))
)

# Safe amount conversion
transactions_clean = transactions_clean.withColumn(
    "amount",
    expr("try_cast(amount_raw AS DOUBLE)")
)

# Safe date conversion
transactions_clean = transactions_clean.withColumn(
    "txn_date",
    coalesce(
        expr("try_to_timestamp(txn_date_raw, 'yyyy-MM-dd')"),
        expr("try_to_timestamp(txn_date_raw, 'MM-dd-yyyy')"),
        expr("try_to_timestamp(txn_date_raw, 'yyyy/MM/dd')")
    ).cast("date")
)

```

```

transactions_clean.filter(
    col("txn_date").isNull()
).select(
    "txn_id",
    "account_id",
    "txn_date_raw",
    "amount_raw"
).show(truncate=False)

```

```

+-----+-----+-----+-----+
|txn_id  |account_id|txn_date_raw|amount_raw|
+-----+-----+-----+-----+
|TXN900002|ACC0010  |NULL        |12000.0   |

```

```
+-----+-----+-----+-----+
```

✓ Step 10: Remove Invalid Transactions

```
transactions_silver = (
    transactions_clean
    .filter(col("txn_id").isNotNull())
    .filter(col("account_id").isNotNull())
    .filter(col("txn_date").isNotNull())
    .filter(col("amount").isNotNull())
    .filter(col("amount") >= 0)
    .dropDuplicates(["txn_id"])
    .select("txn_id", "account_id", "txn_date", "amount", "merchant")
)
```

```
print("Transactions before cleaning:", transactions.count())
print("Transactions after cleaning :", transactions_silver.count())
```

```
Transactions before cleaning: 20011
Transactions after cleaning : 20004
```

```
from pyspark.sql.functions import date_format

display(
    transactions_silver.select(
        col("txn_id"),
        col("account_id"),
        date_format(col("txn_date"), "yyyy-MM-dd").alias("txn_date"),
        col("amount"),
        col("merchant")
    ).head(5)
)
```

```
[Row(txn_id='TXN000001', account_id='ACC0065', txn_date='2025-06-10',
amount=45458.55, merchant='Flipkart'),
 Row(txn_id='TXN000002', account_id='ACC0258', txn_date='2025-06-14',
amount=25336.95, merchant='Swiggy'),
 Row(txn_id='TXN000003', account_id='ACC0088', txn_date='2025-06-18',
amount=23600.65, merchant='BigBasket')]
```

✓ Step 11: Check Transaction Quality

```
print(
    "Null transaction IDs:",
    transactions_silver.filter(col("txn_id").isNull()).count()
)
```

```

print(
    "Null account IDs:",
    transactions_silver.filter(col("account_id").isNull()).count()
)

print(
    "Null amounts:",
    transactions_silver.filter(col("amount").isNull()).count()
)

print(
    "Negative amounts:",
    transactions_silver.filter(col("amount") < 0).count()
)

```

```

Null transaction IDs: 0
Null account IDs: 0
Null amounts: 0
Negative amounts: 0

```

✓ Step 12: Remove Orphan Transactions

```

orphan_transactions = transactions_silver.join(
    accounts_silver.select("account_id"),
    on="account_id",
    how="left_anti"
)

print("Orphan transactions:", orphan_transactions.count())

orphan_transactions.show()

```

```

Orphan transactions: 1
+-----+-----+-----+-----+-----+
|account_id|  txn_id|  txn_date|amount|merchant|
+-----+-----+-----+-----+-----+
|   ACC9999|TXN900009|2025-02-14|1500.0|   Uber|
+-----+-----+-----+-----+-----+

```

✓ Step 13: Clean Fraud Watchlist

```

# Step 13: Clean Fraud Watchlist (FIXED)

from pyspark.sql.functions import col, trim, lit, coalesce, expr, regexp_replace

# Guard against ANSI mode throwing on bad casts elsewhere in the notebook too

```



```

spark.conf.set("spark.sql.ansi.enabled", "false")

fraud_clean = (
    fraud_watchlist
    .withColumn("account_id", trim(col("account_id")))
    .withColumn("fraud_type", trim(col("fraud_type")))
    .withColumn("flagged_date_raw", trim(col("flagged_date")))
)

# Normalize "10-Feb-2025" -> "10-02-2025" using a clean, complete month map
month_map = {
    "Jan": "01", "Feb": "02", "Mar": "03", "Apr": "04",
    "May": "05", "Jun": "06", "Jul": "07", "Aug": "08",
    "Sep": "09", "Oct": "10", "Nov": "11", "Dec": "12",
}

normalized_date = col("flagged_date_raw")
for abbr, num in month_map.items():
    normalized_date = regexp_replace(normalized_date, abbr, num)

fraud_clean = fraud_clean.withColumn("flagged_date_normalized", normalized_date)

# Use expr()-wrapped try_to_timestamp – this is what actually returns NULL
# instead of throwing, unlike the bare python try_to_timestamp() function.
fraud_clean = fraud_clean.withColumn(
    "flagged_date",
    coalesce(
        expr("try_to_timestamp(flagged_date_raw, 'yyyy-MM-dd')"),
        expr("try_to_timestamp(flagged_date_normalized, 'dd-MM-yyyy')"),
        expr("try_to_timestamp(flagged_date_raw, 'MM-dd-yyyy')")
    ).cast("date")
)

fraud_silver = (
    fraud_clean
    .filter(col("account_id").isNotNull() & (col("account_id") != ""))
    .filter(col("fraud_type").isNotNull() & (col("fraud_type") != ""))
    .filter(col("flagged_date").isNotNull())
    .dropDuplicates(["account_id"])
    .select("account_id", "fraud_type", "flagged_date")
)

print("Fraud watchlist before cleaning:", fraud_watchlist.count())
print("Fraud watchlist after cleaning :", fraud_silver.count())

```

```

Fraud watchlist before cleaning: 46
Fraud watchlist after cleaning : 44

```

▼ Step 14: Remove Orphan Fraud Accounts

```
# Step 14: Remove Orphan Fraud Accounts
orphan_fraud = fraud_silver.join(
    accounts_silver.select("account_id"),
    on="account_id",
    how="left_anti"
)
print("Orphan fraud accounts:", orphan_fraud.count())
orphan_fraud.show()

fraud_silver = fraud_silver.join(
    accounts_silver.select("account_id"),
    on="account_id",
    how="left_semi"
)
```

```
Orphan fraud accounts: 1
+-----+-----+-----+
|account_id|      fraud_type|flagged_date|
+-----+-----+-----+
|  ACC9998|Account Takeover|  2025-03-05|
+-----+-----+-----+
```

✓ Step 15: Enrich Transactions

```
enriched_transactions = transactions_silver.join(
    accounts_silver,
    on="account_id",
    how="left"
)
```

```
enriched_transactions = enriched_transactions.select(
    "txn_id",
    "account_id",
    "customer_name",
    "account_type",
    "credit_limit",
    "branch",
    "txn_date",
    "amount",
    "merchant"
)
```

```
enriched_transactions.show(10, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+
|txn_id  |account_id|customer_name|account_type|credit_limit|branch  |txn_d
```

```
+-----+-----+-----+-----+-----+-----+
|TXN000001|ACC0065|Customer_65|Savings|100000.0|Mumbai|2025-
|TXN000002|ACC0258|Customer_258|Current|100000.0|Chennai|2025-
|TXN000003|ACC0088|Customer_88|Current|50000.0|Delhi|2025-
|TXN000004|ACC0432|Customer_432|Current|100000.0|Chennai|2025-
|TXN000005|ACC0042|Customer_42|Savings|500000.0|Chennai|2025-
|TXN000006|ACC0334|Customer_334|Current|50000.0|Pune|2025-
|TXN000007|ACC0317|Customer_317|Salary|200000.0|Chennai|2025-
|TXN000008|ACC0329|Customer_329|Current|50000.0|Bangalore|2025-
|TXN000009|ACC0153|Customer_153|Salary|200000.0|Mumbai|2025-
|TXN000010|ACC0081|Customer_81|Savings|500000.0|Delhi|2025-
+-----+-----+-----+-----+-----+-----+
```

only showing top 10 rows

✓ Step 16: Final Schema Validation

```
enriched_transactions.printSchema()
```

```
root
|-- txn_id: string (nullable = true)
|-- account_id: string (nullable = true)
|-- customer_name: string (nullable = true)
|-- account_type: string (nullable = true)
|-- credit_limit: double (nullable = true)
|-- branch: string (nullable = true)
|-- txn_date: date (nullable = true)
|-- amount: double (nullable = true)
|-- merchant: string (nullable = true)
```

✓ Step 17: Save Silver Layer

```
import os
```

```
os.makedirs("silver", exist_ok=True)
```

```
accounts_silver.write.mode("overwrite").parquet(
    "silver/accounts"
)
```

```
transactions_silver.write.mode("overwrite").parquet(
    "silver/transactions"
)
```

```
fraud_silver.write.mode("overwrite").parquet(
    "silver/fraud_transactions"
)
```

```

    silver/fraud_watchlist
)

enriched_transactions.write.mode("overwrite").parquet(
    "silver/enriched_transactions"
)

```

✓ Step 18: Final Silver Summary

```

print("===== SILVER LAYER SUMMARY =====")

print("Clean Accounts      :", accounts_silver.count())
print("Clean Transactions   :", transactions_silver.count())
print("Clean Fraud Watchlist:", fraud_silver.count())
print("Enriched Transactions:", enriched_transactions.count())

===== SILVER LAYER SUMMARY =====
Clean Accounts      : 501
Clean Transactions   : 20004
Clean Fraud Watchlist: 43
Enriched Transactions: 20004

```

✓ Step 19: Display Final Data

```
enriched_transactions.show(10, truncate=False)
```

txn_id	account_id	customer_name	account_type	credit_limit	branch	txn_d
TXN000001	ACC0065	Customer_65	Savings	100000.0	Mumbai	2025-
TXN000002	ACC0258	Customer_258	Current	100000.0	Chennai	2025-
TXN000003	ACC0088	Customer_88	Current	50000.0	Delhi	2025-
TXN000004	ACC0432	Customer_432	Current	100000.0	Chennai	2025-
TXN000005	ACC0042	Customer_42	Savings	500000.0	Chennai	2025-
TXN000006	ACC0334	Customer_334	Current	50000.0	Pune	2025-
TXN000007	ACC0317	Customer_317	Salary	200000.0	Chennai	2025-
TXN000008	ACC0329	Customer_329	Current	50000.0	Bangalore	2025-
TXN000009	ACC0153	Customer_153	Salary	200000.0	Mumbai	2025-
TXN000010	ACC0081	Customer_81	Savings	500000.0	Delhi	2025-

only showing top 10 rows

✓ Step 20: Download the Silver Layer Ouput

```

from pyspark.sql import DataFrame
import os

```

```
import shutil

def save_as_csv(df, filename):
    temp_dir = f"temp_{filename}"

    # Write Spark output
    df.coalesce(1).write \
        .mode("overwrite") \
        .option("header", True) \
        .csv(temp_dir)

    # Find generated CSV file
    csv_file = [
        f for f in os.listdir(temp_dir)
        if f.endswith(".csv")
    ][0]

    # Rename to normal CSV filename
    shutil.copy(
        os.path.join(temp_dir, csv_file),
        filename
    )

    # Remove temporary Spark folder
    shutil.rmtree(temp_dir)

    print(f"Created: {filename}.csv")

save_as_csv(accounts_silver, "silver_accounts.csv")
save_as_csv(transactions_silver, "silver_transactions.csv")
save_as_csv(fraud_silver, "silver_fraud_watchlist.csv")
save_as_csv(enriched_transactions, "silver_enriched_transactions.csv")
```

```
Created: silver_accounts.csv.csv
Created: silver_transactions.csv.csv
Created: silver_fraud_watchlist.csv.csv
Created: silver_enriched_transactions.csv.csv
```

```
from google.colab import files

files.download("silver_accounts.csv")
files.download("silver_transactions.csv")
files.download("silver_fraud_watchlist.csv")
files.download("silver_enriched_transactions.csv")
```

Conclusion

The Silver layer successfully cleaned and validated the raw datasets. Null values, duplicate records, invalid dates, invalid amounts, negative values, whitespace and orphan records were handled.

The cleaned transaction data was enriched with account information using an `account_id`-based join. The resulting Silver datasets are ready for fraud classification and business analysis in the Gold layer.